

# Project Technical Documentation

**Production-Grade AI Data Agent & Business Intelligence Platform**  
Built with Streamlit, Groq API (Llama 3.3), and Relational Databases

## 1. Executive Summary & Core Objective

In the modern enterprise landscape, business managers and non-technical decision-makers frequently require instant access to organizational metrics. However, traditional data retrieval introduces friction—requiring either direct knowledge of data manipulation languages (SQL) or reliance on over-burdened data engineering teams.

This project introduces an **AI-Powered Text-to-SQL Analytics Dashboard** that directly addresses this problem. By bridging Advanced Natural Language Processing (NLP) with secure relational database connectors, the application enables users to ask complex business questions in conversational plain English. The backend intelligently builds, cleans, tests, and renders the requested query alongside rich analytical visual graphs within seconds, turning a standard database into a smart conversation partner.

## 2. System Architecture & File Interactions

The platform is engineered using clean, decoupled architectural patterns to isolate state management from visual representation. The interactions are split across four major files:

| File Name   | Layer Designation            | Functional Responsibility                                                                                                               |
|-------------|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| app.py      | Presentation UI Layer        | Coordinates Streamlit server, takes user inputs, manages rendering states, and plots graphs dynamically via Plotly.                     |
| ai_agent.py | Cognitive Intelligence Layer | Ingests user inputs and structural schemas. Communicates with Groq's high-speed LPU infrastructure using zero-shot inference framework. |
| database.py | Infrastructure Engine Layer  | Handles connection persistence, converts queries to Pandas DataFrames, and evaluates security validation matrices.                      |
| schema.sql  | Relational Data Layer        | Defines table architectures, fields, operational primary/foreign constraints, and records mock transactional items.                     |

### 3. Step-by-Step Data Processing Pipeline

When an end-user issues a request through the web frontend interface, the system initiates an optimized 5-stage synchronous operational pipeline:

- **Stage 1: Context Aggregation & Bootstrapping:** The application server starts and checks the physical structure of the database. It captures database table names, distinct column arrays, and field relationships to build an unambiguous schema manifest text file.
- **Stage 2: Context-Aware SQL Generation (Groq Core):** The user's natural language input string is paired with the schema manifest text file. The system packages this data inside an engineering-grade system prompt and dispatches it to the Groq cloud infrastructure. The Llama 3.3 70B model parses the parameters and returns an optimized, precise SQL string with a high accuracy target rate.
- **Stage 3: Active Interception & Security Sanitization:** To protect enterprise resources, the system feeds the returned SQL text into a rigid validation gate. If illegal keywords matching destructive administrative operations (e.g., DROP, DELETE, ALTER, UPDATE) are present, execution is aborted, and a security alert is triggered.
- **Stage 4: Relational Query Execution:** The sanitized query string is handed down to the internal engine framework. The database runs the query, executes complex multi-table joins, gathers data rows, and returns the records into memory as a highly flexible Python Pandas DataFrame object.
- **Stage 5: Intelligent Automatic Visualization:** The UI logic measures the shapes, column types, and data formats inside the dataframe. If categorical groups coexist alongside quantitative numerical values, the system chooses the best fitting chart layout (e.g., Bar, Line, or Scatter) and plots interactive graphs immediately.

### 4. Technical Competencies Highlighted for Recruiters

This application moves far beyond standard template projects because it synthesizes a combination of high-value industry skills:

- **Advanced Prompt Engineering:** Forces LLMs to act as specialized code compilers that strictly emit clean SQL blocks without conversational chit-chat or text formatting errors.
- **Hardware Acceleration Utilization:** Leverages Groq's Language Processing Units (LPUs) to demonstrate knowledge of ultra-low latency inference, high token throughput, and modern cost-effective open-source alternatives.
- **Data Security Guardrails:** Solves a critical commercial problem by blocking injection vectors and enforcing read-only operations directly at the application tier.
- **Full-Stack Data Engineering:** Bridges relational schemas, database engines, data formatting, statistical chart creation, and UI orchestration together in one complete system.

---

**Portfolio Disclaimer:** This document acts as technical presentation blueprint material for recruiter evaluation. All architectural principles represent industry-standard engineering models for data agents.